CS2601 Linear and Convex Optimization
Homework 8 Solution

Due: 2021.11.26

For this assignment, you should submit a **single** pdf file as well as your source code (.py or .ipynb files). The pdf file should include all necessary figures, the outputs of your Python code, and your answers to the questions. Do NOT submit your figures in separate files. Your answers in any of the .py or .ipynb files will NOT be graded.

1. In this problem, you will use Newton's method to solve Problem 1 of Homework 7, i.e. (9.20) of Boyd and Vandenberghe,

$$\min_{x_1,x_2} \; f(x_1,x_2) = e^{x_1+3x_2-0.1} + e^{x_1-3x_2-0.1} + e^{-x_1-0.1} \tag{1}$$

First implement the pure Newton's method in `newton.py`.

(a). Solve (1) numerically using your implementation of Newton's method. Use the initial point $\boldsymbol{x}_0 = (-1.5, 1)^T$. Report the solution and the number of iterations. Plot the trajectory of $\boldsymbol{x}_k$ and the gap $f(\boldsymbol{x}_k) - f(\boldsymbol{x}^*)$.

(b). Repeat (a) for the initial point $\boldsymbol{x}_0 = (1.5, 1)^T$.

Proof. (a). For the plots, see Figure 1.

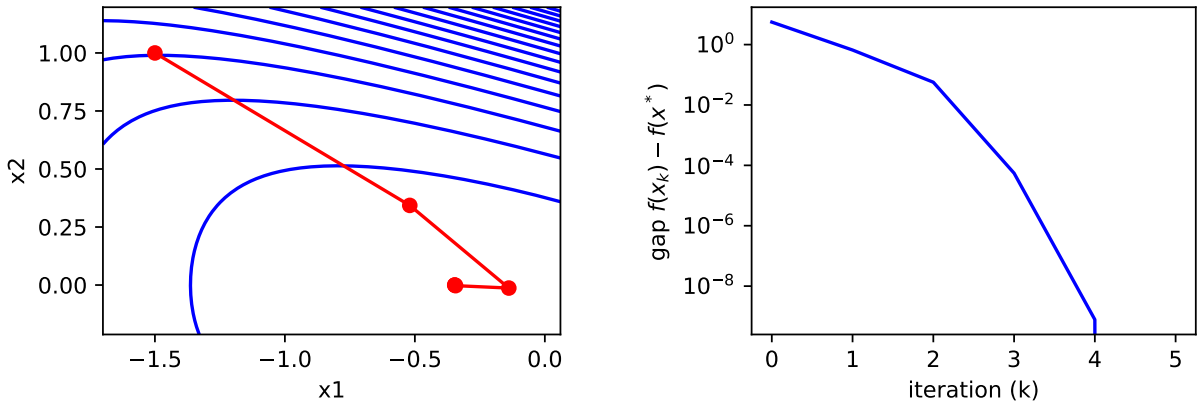
```
Newton's method
  number of iterations: 5
  solution: [-3.46573590e-01 -1.41533674e-10]
  value: 2.5592666966582156
```

(b). For the plots, see Figure 2.

```
Newton's method
  number of iterations: 8
  solution: [-3.46573573e-01 -1.13424622e-08]
  value: 2.559266696658217
```

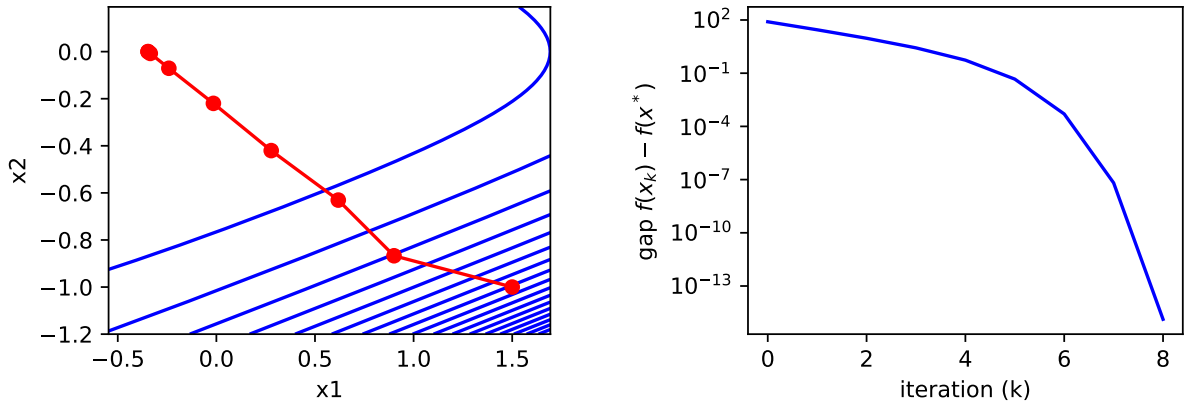
□

A possible implementation.



(a) trajectory of $\boldsymbol{x}_k$ **(b) gap**

Figure 1: $\boldsymbol{x}_0 = (-1.5, 1)^T$



(a) trajectory of $\boldsymbol{x}_k$ **(b) $f(\boldsymbol{x}_k) - f(\boldsymbol{x}^*)$**

Figure 2: $\boldsymbol{x}_0 = (1.5, 1)^T$

```
def newton(fp, fpp, x0, tol=1e-5, maxiter=100000):
    x_traces = [np.array(x0)]
    x = np.array(x0)

    for k in range(maxiter):
        grad = fp(x)

        if np.linalg.norm(grad) < tol:
            break

        x -= np.linalg.solve(fpp(x), grad)

        x_traces.append(np.array(x))

    return x_traces
```

2. Logistic regression. First implement the damped Newton's method in `newton.py`. Then your implementation of damped Newton's method to solve Problem 3 of Homework 6,

$$\min_{\boldsymbol{w}} \ f(\boldsymbol{w}) = \sum_{i=1}^{m} \log(1 + e^{-y_i \boldsymbol{x}_i^T \boldsymbol{w}})$$

Recall

$$\nabla f(\boldsymbol{w}) = -\sum_{i=1}^{m} [1 - \sigma(y_i \boldsymbol{x}_i^T \boldsymbol{w})] y_i \boldsymbol{x}_i$$

(a). Show the Hessian of f is

$$\nabla^2 f(\boldsymbol{w}) = \sum_{i=1}^{m} \sigma'(y_i \boldsymbol{x}_i^T \boldsymbol{w}) \boldsymbol{x}_i \boldsymbol{x}_i^T$$

Note that here $\boldsymbol{x}_i$ is considered as a column vector, but in the implementation $\boldsymbol{x}_i$ is stored as the i-th row of the matrix $\boldsymbol{X}$, i.e. $\boldsymbol{X}^T = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m]$.

(b). Find the optimal $\boldsymbol{w}^*$ using your implementation of damped Newton's method. Use $\alpha = 0.1$ and $\beta = 0.7$ and initial point $\boldsymbol{w}_0 = (1, 1, 0)^T$. Report the solution, the number of iterations in the outer loop and the total number of iterations in the inner loop. Plot the gap $f(\boldsymbol{x}_k) - f(\boldsymbol{x}^*)$ and the step sizes t_k.

Hint: You may consider using the broadcasting mechanism of python. Also note if $\boldsymbol{X}^T = (\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m)^T$ and $\mathbb{Z}^T = (\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_m)$, then

$$\sum_{i=1}^{m} \boldsymbol{z}_i \boldsymbol{x}_i^T = [\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_m] \begin{bmatrix} \boldsymbol{x}_1^T \\ \vdots \\ \boldsymbol{x}_m^T \end{bmatrix} = \mathbf{Z}^T \boldsymbol{X}$$

(c). What happens if you use (pure) Newton's method with $\boldsymbol{w}_0 = (1, 1, 0)^T$?

Solution. (a). Let $g_j(\boldsymbol{w}) = [\nabla f(\boldsymbol{w})]_i = -\sum_{i=1}^{m} [1 - \sigma(y_i \boldsymbol{x}_i^T \boldsymbol{w})] y_i x_{ij}$.

$$\frac{\partial g_j(\boldsymbol{w})}{\partial w_k} = \sum_{i=1}^{m} \sigma'(y_i \boldsymbol{x}_i^T \boldsymbol{w})] y_i y_i x_{ik} x_{ij} = \sum_{i=1}^{m} \sigma'(y_i \boldsymbol{x}_i^T \boldsymbol{w})] x_{ij} x_{ik}$$

so

$$\nabla^2 f(\boldsymbol{w}) = \sum_{i=1}^{m} \sigma'(y_i \boldsymbol{x}_i^T \boldsymbol{w}) \boldsymbol{x}_i \boldsymbol{x}_i^T$$

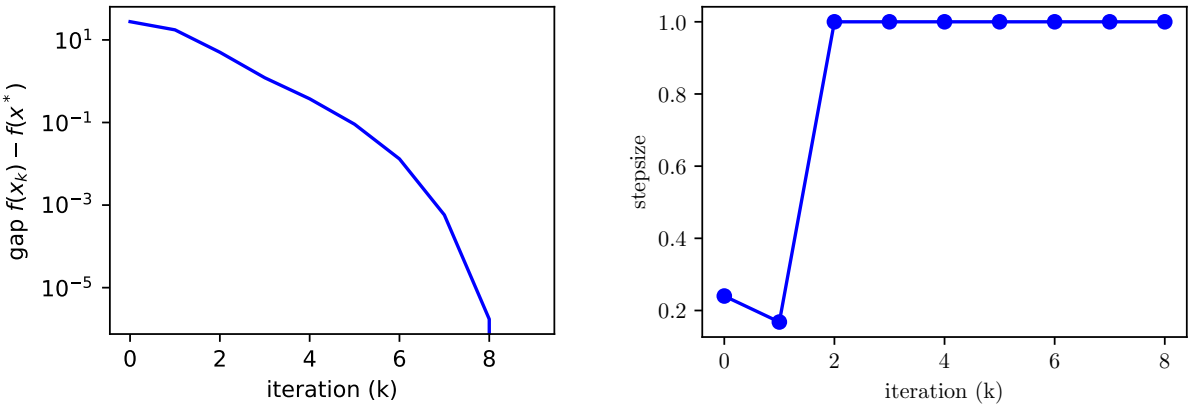
(b). A implementation of `damped_newton` can be obtained by modifying `gd_amijo`. A possible implementation of `fpp` is the following.

```
def fpp(w):
    SX = sigmoid_p(Xy@w).reshape((-1,1)) * X
    return SX.T@X
```

The output is given below, and the plots in Figure 3.

```
Damped Newton's method
```

```
number of iterations in outer loop: 9
total number of iterations in inner loop: 9
solution: [-1.47021306  4.44400878 -4.3758784 ]
value: 2.876681099986132
```



(a) trajectory of x_k **(b) gap**

Figure 3: Damped Newton's method with $w_0 = (1, 1, 0)^T$

(c). Newton's method diverges.

□

3. Consider the optimization problem $\min_x f(x)$, where $f : \mathbb{R} \to \mathbb{R}$ is given by $f(x) = (x-a)^4$, and $a \in \mathbb{R}$ is a constant.

(a). Find an explicit expression for the Newton step.

(b). Let x_k be the sequence of iterates generated by Newton's method. Let $y_k = |x_k - a|$ be the error between the k-th iterate and the optimal solution. Show that $y_{k+1} = \frac{2}{3} y_k$

(c). Conclude $|x_k - a|$ decays to zero exponentially, i.e. x_k converges exponentially to a.

Proof. (a). $f'(x) = 4(x-a)^3$, $f''(x) = 12(x-a)^2$.

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)} = x_k - \frac{4(x_k-a)^3}{12(x_k-a)^2} = x_k - \frac{1}{3}(x_k - a) = \frac{2}{3}x_k + \frac{1}{3}a$$

(b). Since

$$x_{k+1} = x_k - \frac{1}{3}(x_k - a)$$

we obtain

$$x_{k+1} - a = (x_k - a) - \frac{1}{3}(x_k - a) = \frac{2}{3}(x_k - a)$$

Taking the absolute values,

$$y_{k+1} = \frac{2}{3} y_k$$

(c).

$$|x_k - a| = y_k = \left(\frac{2}{3}\right)^k y_0$$

converges exponentially to 0.

□

Remark. Note that the convergence rate here is only exponential no matter how close x_0 is to $x^* = a$, while the rate given by the theorem in Lecture 10 is doubly exponential, which is much faster, at least when x_0 is close enough to x^*. This is because $(x-a)^4$ is not strongly convex, which does not satisfy the assumptions of the theorem.

4. Complete the implementation of the soft-thresholding operator and ISTA in `ista.py` for solving Lasso in penalized form,

$$f(\boldsymbol{w}) = \frac{1}{2}\|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}\|_2^2 + \lambda\|\boldsymbol{w}\|_1$$

We will explore the effect of λ on the number of zeros in the solution $\boldsymbol{w}^*$.

(a). Reproduce the result on slide 10 of §8, with

$$\boldsymbol{X} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad \boldsymbol{y} = \begin{bmatrix} 3 \\ 2 \\ 2 \end{bmatrix}, \quad \lambda = 2$$

step size $t = 0.1$, and initial point $\boldsymbol{w}_0 = (0.5, 0.5)^T$. Report the solution and the number of iterations. Plot the trajectory of $\boldsymbol{w}_k$ and the gap $f(\boldsymbol{w}_k) - f(\boldsymbol{x}^*)$. Use the solution you find in place of $f(\boldsymbol{x}^*)$.

(b). Redo part (b) with $\lambda = 1$. Do you get zeros in $\boldsymbol{w}^*$?

(c). Redo part (b) with $\lambda = 6$. How many zeros do you get in $\boldsymbol{w}^*$?

Remark. There may be numerical errors in the $\boldsymbol{w}^*$ you obtain, but you can safely guess the exact $\boldsymbol{w}^*$ from the numerical solutions. If you want to verify your guess (you don't have to for this assignment), you can use the following condition: $\boldsymbol{w}$ is optimal iff

$$(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})_i + \lambda \operatorname{sgn}(w_i) = 0 \text{ if } w_i \neq 0$$

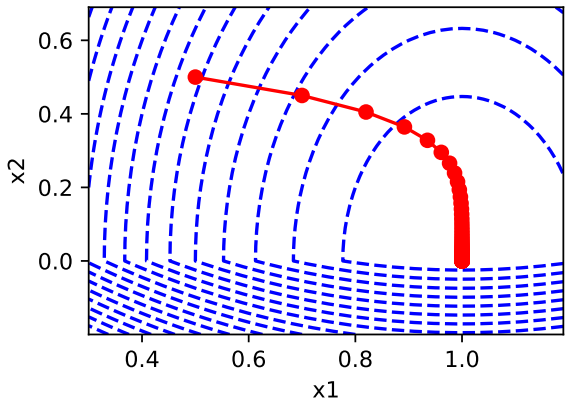
and

$$-\lambda \leq (\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})_i \leq \lambda \text{ if } w_i = 0$$

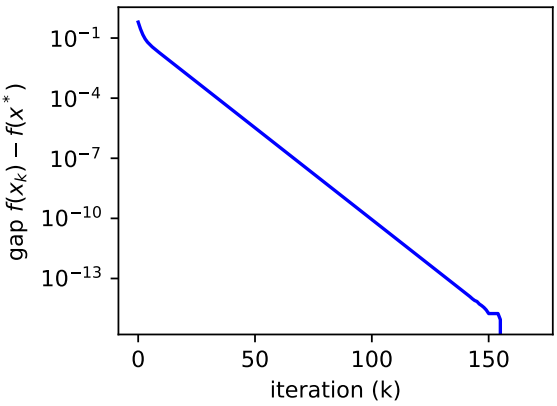
Proof. (a). For the plots, see Figure 4.

```
lambda =  2
number of iterations: 169
solution: [1.00000000e+00 9.24600449e-09]
value: 6.5
```

(b). For the plots, see Figure 5. No zeros in $\boldsymbol{w}^*$.



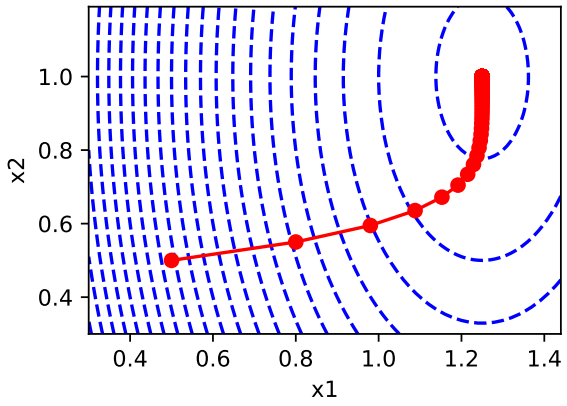
(a) trajectory of w_k



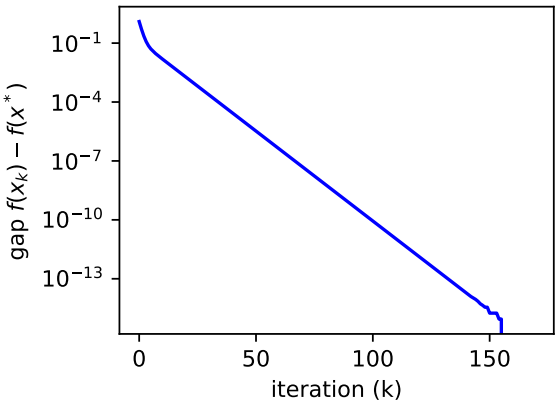
(b) gap

Figure 4: $\lambda = 2$

```
lambda =  1
number of iterations: 169
solution: [1.25       0.99999999]
value: 4.875
```



(a) trajectory of w_k



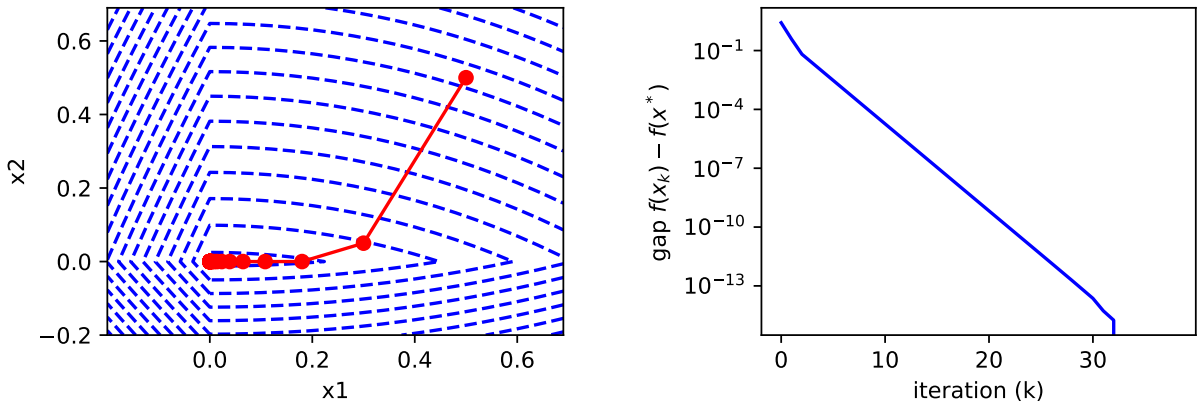
(b) gap

Figure 5: $\lambda = 1$

(c). For the plots, see Figure 6. Two zeros in $\boldsymbol{w}^*$.

```
lambda =  6
number of iterations: 38
solution: [1.85659632e-09 0.00000000e+00]
value: 8.500000000000002
```

□



(a) trajectory of w_k **(b) gap**

Figure 6: $\lambda = 6$